UNIVERSITÀ DELLA CALABRIA

Algorithmic Game Theory — a.a. 2026/2027 — Sebastiano A. Piccolo

Appunti in italiano, parte di `main.pdf` — 5 novembre 2026

LEZIONE 6

Giochi coalizionali: aspetti computazionali

TEORIA

La lezione 5 si chiudeva con un problema: la funzione caratteristica ha 2^n valori e il valore di Shapley li usa tutti. Questa lezione mostra tre modi per scalare a molti giocatori. Il percorso:

1. il problema computazionale;
2. una classe speciale di giochi: il **gioco dell'aeroporto**;
3. i **giochi su sottografo indotto** (ISG);
4. una rappresentazione compatta universale: le **MC-Nets**;
5. l'**approssimazione Monte Carlo** del valore di Shapley.

6.1 Il problema computazionale

Un gioco coalizionale è rappresentato da una funzione caratteristica $v : 2^N \to \mathbb{R}$, che ha un valore per ogni sottoinsieme di giocatori: la sua dimensione è **esponenziale** nel numero di giocatori. Già con tre giocatori (il gioco del gelato della lezione 5) servono otto numeri:

$$v(\emptyset) = v(\{A\}) = v(\{B\}) = v(\{C\}) = 0,$$
$$v(\{A,B\}) = v(\{A,C\}) = 750, \quad v(\{B,C\}) = 0, \quad v(\{A,B,C\}) = 1000.$$

Di conseguenza anche i concetti di soluzione richiedono un numero esponenziale di calcoli. La formula di Shapley somma su tutte le permutazioni:

$$\phi(i,v) = \frac{1}{|N|!} \sum_{\pi \in \Pi_N} \Big(v\big(B(\pi,i) \cup \{i\}\big) - v\big(B(\pi,i)\big) \Big).$$

La versione sui sottoinsiemi scende a 2^{n-1} termini per giocatore, ma resta esponenziale.

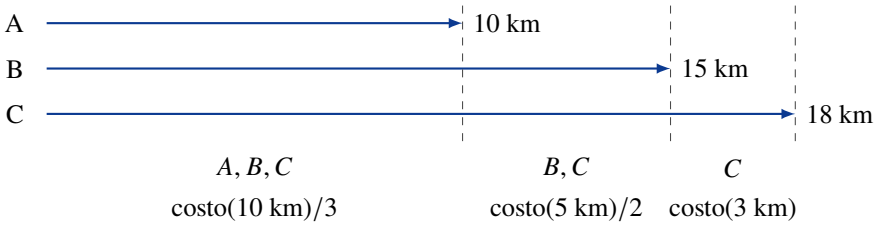
Tre strade.

Strada	Che cosa si ottiene
Classi speciali di giochi	formule chiuse per il valore di Shapley, calcolabili in tempo polinomiale (gioco dell'aeroporto)
Rappresentazione compatta	funzione caratteristica in spazio polinomiale e valore di Shapley in tempo polinomiale (ISG, MC-Nets)
Approssimazione Monte Carlo	tempo polinomiale per *qualunque* gioco, con garanzie sull'errore

6.2 Il gioco dell'aeroporto

Ci sono n compagnie aeree. Ognuna ha bisogno di una pista di una certa lunghezza per i propri aerei. Una pista può essere condivisa, quindi conviene costruirne *una sola*, abbastanza lunga per tutti, e dividerne il costo. Come lo si divide in modo equo?

Intuizione. La compagnia A ha bisogno di 10 km, B di 15 km, C di 18 km. Si costruisce una pista di 18 km, ma i suoi tratti servono a gruppi diversi:



I primi 10 km servono a tutti e tre, quindi il loro costo si divide per 3. I 5 km successivi servono solo a B e C, che se li dividono in due. Gli ultimi 3 km servono solo a C, che li paga da solo.

Definizione 6.1 (Gioco dell'aeroporto). I giocatori $N = \{1, \dots, n\}$ hanno costi ordinati $c_1 < c_2 < \cdots < c_n$, dove c_i è il costo della pista che serve al giocatore i. La funzione caratteristica è

$$v(S) = \max_{i \in S} c_i \qquad \forall S \subseteq N :$$

una coalizione paga la pista del suo membro più esigente. È un gioco di **costo**: $v(S)$ è una spesa da ripartire, non un guadagno.

Teorema 6.2 (Littlechild e Owen, 1973). Il valore di Shapley del gioco dell'aeroporto è

$$\phi_i = \sum_{j=1}^{i} \frac{c_j - c_{j-1}}{n - j + 1} \qquad \forall i \in N, \qquad c_0 = 0.$$

Lettura della formula. Il tratto di pista tra c_{j-1} e c_j costa $c_j - c_{j-1}$ e serve ai giocatori $j, j+1, \dots, n$, che sono $n - j + 1$: se lo dividono in parti uguali. Il giocatore i paga la sua quota di ogni tratto fino al proprio, cioè per $j = 1, \dots, i$.

Dimostrazione. Si definisce $P_j = \{j, \dots, n\}$, l'insieme dei giocatori che hanno bisogno del tratto j, e per ogni j il gioco

$$v_j(S) = \begin{cases} 0 & \text{se } S \cap P_j = \emptyset, \\ c_j - c_{j-1} & \text{se } S \cap P_j \neq \emptyset. \end{cases}$$

Una coalizione S con membro più esigente k interseca esattamente $P_1, \dots, P_k$, quindi $\sum_j v_j(S) = \sum_{j=1}^{k}(c_j - c_{j-1}) = c_k = v(S)$: il gioco dell'aeroporto è la somma dei giochi v_j. In v_j i giocatori fuori da P_j sono **nulli** (non cambiano mai il valore), quelli dentro P_j sono **simmetrici**; per efficienza si dividono in parti uguali $c_j - c_{j-1}$:

$$\phi_i(v_j) = \begin{cases} \dfrac{c_j - c_{j-1}}{n - j + 1} & \text{se } i \in P_j \text{ (cioè } j \leq i), \\ 0 & \text{altrimenti.} \end{cases}$$

Per **additività**, $\phi_i(v) = \sum_j \phi_i(v_j) = \sum_{j=1}^{i} \frac{c_j - c_{j-1}}{n-j+1}$. □

Osservazione (gli assiomi come strumento di calcolo). La dimostrazione non tocca mai le $n!$ permutazioni: scompone il gioco in pezzi semplici e usa i quattro assiomi della lezione 5 (nullo, simmetria, efficienza, additività). È lo stesso trucco che rende veloci ISG e MC-Nets: *additività* vuol dire che basta calcolare Shapley su ogni pezzo e sommare.

Esempio (quattro compagnie, costi $(8, 11, 13, 18)$**).**

	Tratto 1	Tratto 2	Tratto 3	Tratto 4	**Shapley**
Costo marginale	8	3	2	5	
Quota di P1	2				**2**
Quota di P2	2	1			**3**
Quota di P3	2	1	1		**4**
Quota di P4	2	1	1	5	**9**

Il primo tratto (8) si divide fra quattro, il secondo ($11 - 8 = 3$) fra tre, il terzo ($13 - 11 = 2$) fra due, l'ultimo ($18 - 13 = 5$) lo paga solo P4. Totale $2 + 3 + 4 + 9 = 18 = v(N)$.

Complessità. La formula come scritta richiede $O(n^2)$ operazioni (una somma di i termini per ogni giocatore). Ma ogni riga della tabella è la riga precedente più un termine: con una somma cumulativa si scende a $O(n)$, più $O(n \log n)$ se i costi vanno ordinati. Contro $O(n \cdot 2^n)$ della formula generale.

Applicazioni. Car sharing e ride sharing, capacità di banda in una rete, una condotta d'acqua o un oleodotto, spazio di archiviazione in cloud con utenti che chiedono quote diverse, livelli di abbonamento a un software.

Osservazione (ricetta per riconoscere un gioco dell'aeroporto). Servono quattro ingredienti:

- una sola struttura o risorsa **condivisa**;
- una struttura **annidata**: chi chiede di più ha bisogno anche di tutto ciò che serve a chi chiede di meno;

- il costo **cresce** con la richiesta;
- l'obiettivo è dividere i costi in modo **equo**.

Approfondimento (il valore di Shapley dell'aeroporto è nel core). Nel gioco dell'aeroporto una coalizione S con membro più esigente k paga, con Shapley, $\sum_{i\in S}\phi_i \le$ la somma delle quote di tutti i giocatori fino a k, cioè al più $c_k = v(S)$. Nessun gruppo pagherebbe meno costruendo la pista da solo: la ripartizione è anche *stabile*. In un gioco di costo la condizione del core ha il verso opposto rispetto alla lezione 5: $\sum_{i\in S} x_i \le v(S)$. Il gioco è infatti *concavo* (per i costi), e per questi giochi Shapley sta sempre nel core.

6.3 Giochi su sottografo indotto

Definizione 6.3 (Induced subgraph game, ISG). I giocatori sono i nodi di un grafo pesato. Un arco $\{i,j\}$ con peso w_{ij} rappresenta il valore della coppia; un cappio con peso w_{ii} il valore del giocatore da solo. Il valore di una coalizione è la somma dei pesi degli archi che hanno *entrambi* gli estremi nella coalizione, cioè il peso del sottografo indotto da C:

$$v(C) = \sum_{\{i,j\}\subseteq C} w_{ij}.$$

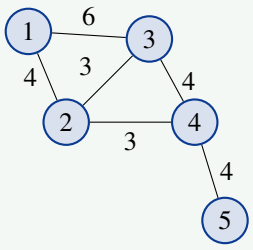
Teorema 6.4 (Shapley negli ISG).

$$\phi_i = w_{ii} + \frac{1}{2}\sum_{j\in\Gamma(i)} w_{ij},$$

dove $\Gamma(i)$ è l'insieme dei vicini di i.

Perché. Di nuovo additività: ogni arco $\{i,j\}$ è un gioco a sé, in cui una coalizione vale w_{ij} se contiene sia i sia j, altrimenti 0. Lì i e j sono simmetrici e tutti gli altri sono nulli, quindi si dividono w_{ij} a metà. Il cappio è un gioco in cui conta solo i, che si prende tutto w_{ii}.

Esempio (ISG con cinque giocatori).



Valori di alcune coalizioni:

$$v(\{1,2,4\}) = 4+3 = 7,$$
$$v(\{1,2,5\}) = 4,$$
$$v(\{1,2,3,4\}) = 4+6+3+4+3 = 20.$$

Valori di Shapley (nessun cappio):

$$\phi_1 = \tfrac{1}{2}(6+4) = 5, \qquad \phi_2 = \tfrac{1}{2}(4+3+3) = 5,$$
$$\phi_3 = \tfrac{1}{2}(6+3+4) = 6{,}5, \qquad \phi_4 = \tfrac{1}{2}(4+3+4) = 5{,}5,$$
$$\phi_5 = \tfrac{1}{2}\cdot 4 = 2.$$

Controllo di efficienza: $5+5+6{,}5+5{,}5+2 = 24$, la somma di tutti i pesi.

Applicazioni. Gli ISG modellano reti sociali e relazioni fra giocatori: pesi **positivi** per le sinergie, pesi **negativi** per i conflitti. Sono anche un modello naturale per la formazione di coalizioni, che si possono cercare con tecniche di clustering di grafi (modularità, partizionamento spettrale con il vettore di Fiedler, conduttanza, Infomap, Walktrap, modelli a blocchi stocastici).

Attenzione (gli ISG non rappresentano tutti i giochi). Il valore di una coalizione è determinato dalle sole coppie: non c'è modo di esprimere un effetto che compare solo quando ci sono *tre* giocatori insieme. Esempio dal notebook: il gioco dell'aeroporto con costi $(5, 7, 10)$. Singoli e coppie fissano i pesi ($w_{AA} = 5$, $w_{BB} = 7$, $w_{CC} = 10$, $w_{AB} = -5$, $w_{AC} = -5$, $w_{BC} = -7$), ma allora l'ISG darebbe $v(\{A, B, C\}) = 5$, mentre il valore vero è 10.

6.4 Rappresentazioni compatte

Una rappresentazione compatta riduce la memoria occupata dalla funzione caratteristica, di solito usando una struttura di rete. Il prezzo:

- il valore di una coalizione non si legge più in $O(1)$ da una tabella, ma si *calcola* in tempo polinomiale (sommando pesi);
- in cambio, sfruttando l'additività del valore di Shapley, anche Shapley si calcola in tempo polinomiale.

Gli ISG sono già una rappresentazione compatta: n nodi e al più $n(n+1)/2$ archi (cappi compresi) invece di 2^n valori. Le MC-Nets la generalizzano fino a rappresentare qualunque gioco.

6.4.1 Marginal Contribution Nets

Definizione 6.5 (MC-Net). Una MC-Net rappresenta la funzione caratteristica come un insieme di **regole**

$$\text{pattern} \longrightarrow \text{valore},$$

dove il pattern è una formula booleana sui giocatori e il valore è il **contributo marginale** del pattern. Nella forma base il pattern è una congiunzione $\{a \wedge b \wedge \cdots \wedge c\}$, e la regola **si applica** a S se S contiene tutti i giocatori del pattern. Il valore di una coalizione è la somma dei valori delle regole che si applicano:

$$v(S) = \sum_{\varphi \to x \,:\, \varphi \subseteq S} x.$$

Se i valori possono essere sia positivi sia negativi, già le congiunzioni bastano a rappresentare *qualunque* gioco.

Esempio (due regole).

$$\{a \wedge b\} \to 5, \qquad \{b\} \to 2.$$

- $v(\{a\}) = 0$: nessuna regola si applica ($\{a\}$ non contiene b);
- $v(\{b\}) = 2$: si applica solo la seconda;
- $v(\{a, b\}) = 5 + 2 = 7$: si applicano entrambe.

ISG come MC-Net. Ogni arco $\{i, j\}$ di peso w diventa la regola $\{i \wedge j\} \to w$ e ogni cappio diventa $\{i\} \to w_{ii}$. Il grafo dell'esempio precedente diventa

$$\{1 \wedge 3\} \to 6, \quad \{1 \wedge 2\} \to 4, \quad \{2 \wedge 3\} \to 3, \quad \{2 \wedge 4\} \to 3, \quad \{3 \wedge 4\} \to 4, \quad \{4 \wedge 5\} \to 4.$$

Una MC-Net è in sostanza un **ipergrafo**: ogni regola è un iperarco che collega tutti i giocatori del pattern, pesato con il suo valore.

Attenzione (errore nella slide "Induced subgraph games as MC-Nets"). La slide scrive $\{4 \wedge 5\} \to 5$, ma nel grafo l'arco $\{4, 5\}$ pesa 4. Con la regola della slide verrebbero $\phi_4 = 6$, $\phi_5 = 2{,}5$ e $v(N) = 25$, invece di 5,5, 2 e 24 (verificato al calcolatore).

Teorema 6.6 (Shapley nelle MC-Nets con congiunzioni).

$$\phi_i = \sum_{\varphi \to x \,:\, i \in \varphi} \frac{x}{|\varphi|}.$$

Ogni regola è un gioco a sé in cui i giocatori del pattern sono simmetrici e gli altri nulli: il valore x si divide in parti uguali fra i $|\varphi|$ giocatori del pattern. Poi additività. Nell'esempio a due regole:

$$\phi_a = \frac{5}{2} = 2{,}5, \qquad \phi_b = \frac{5}{2} + \frac{2}{1} = 4{,}5,$$

e $2{,}5 + 4{,}5 = 7 = v(\{a, b\})$.

Approfondimento (i valori delle regole sono i dividendi di Harsanyi). Per un gioco qualsiasi i valori delle regole con congiunzioni si ricavano con l'inclusione-esclusione (trasformata di Möbius):

$$d(S) = \sum_{T \subseteq S} (-1)^{|S|-|T|}\, v(T),$$

e la MC-Net $\{S \to d(S)\}$ rappresenta esattamente v. È per questo che le congiunzioni con valori di segno qualunque sono complete. La formula di Shapley qui sopra dice allora che ogni giocatore riceve una quota uguale di ogni dividendo delle coalizioni a cui appartiene.
Ma "completa" non vuol dire "compatta". Per il gioco dell'aeroporto il calcolo dà $d(S) = (-1)^{|S|+1} \min_{i \in S} c_i$, mai zero: servono tutte le $2^n - 1$ regole (verificato fino a $n = 10$: 1023 regole). Con costi $(5, 7, 10)$ si ottiene proprio la MC-Net del notebook, con valori $5, 7, 10, -5, -5, -7, +5$.

6.4.2 MC-Nets con formule generali

Le regole possono contenere formule logiche arbitrarie, con letterali **positivi** e **negativi**. Una regola $\varphi \to x$ si applica a S se S **soddisfa** φ, scritto $S \models \varphi$: assegnando "vero" ai giocatori in S e "falso" a tutti gli altri, la formula risulta vera. Il valore di una coalizione è

$$v(S) = \sum_{\varphi \to x \,:\, S \models \varphi} x.$$

Esempio (MC-Net con un letterale negativo).

$$x_1 \wedge x_2 \to 5 \qquad x_2 \to 2 \qquad x_3 \to 4 \qquad x_2 \wedge \neg x_3 \to -2$$

S	regole applicate	$v(S)$
$\{x_1\}$	nessuna	0
$\{x_2\}$	2 e 4	$2-2=0$
$\{x_3\}$	3	4
$\{x_1,x_2\}$	1, 2, 4	$5+2-2=5$
$\{x_1,x_3\}$	3	4
$\{x_2,x_3\}$	2, 3	$2+4=6$
$\{x_1,x_2,x_3\}$	1, 2, 3	$5+2+4=11$

La regola 4 si applica solo quando c'è x_2 *e manca* x_3.

Teorema 6.7 (Shapley nelle MC-Nets generali). Per una regola $\varphi \to x$ con p letterali positivi e n letterali negativi, ogni giocatore i che compare in forma positiva e ogni giocatore j che compare in forma negativa ricevono

$$\phi_i = \frac{(p-1)!\,n!}{(p+n)!}\,x = \frac{x}{p\binom{p+n}{n}}, \qquad \phi_j = \frac{p!\,(n-1)!}{(p+n)!}\,(-x) = -\frac{x}{n\binom{p+n}{p}}.$$

Gli altri giocatori sono nulli per quella regola. Per additività il valore di Shapley è la somma su tutte le regole.

Da dove vengono i coefficienti. Si guardano solo i $p+n$ giocatori della regola, perché gli altri non contano. Un giocatore positivo i "accende" la regola se arriva *dopo* tutti gli altri positivi e *prima* di tutti i negativi; la frazione di ordini in cui succede è $\frac{(p-1)!\,n!}{(p+n)!}$. Un negativo j "spegne" la regola (contributo $-x$) se arriva dopo tutti i positivi e prima degli altri negativi: frazione $\frac{p!\,(n-1)!}{(p+n)!}$. Senza negativi ($n=0$) si ritrova x/p.

Esempio (Shapley nella MC-Net con letterale negativo).

$$\phi_{x_1} = \tfrac{1!\,0!}{2!}\cdot 5 = 2{,}5 \qquad \text{(solo regola 1)}$$
$$\phi_{x_2} = \tfrac{1!\,0!}{2!}\cdot 5 + 2 + \tfrac{1!\,0!}{2!}\cdot(-2) = 2{,}5+2-1 = 3{,}5 \qquad \text{(regole 1, 2, 4)}$$
$$\phi_{x_3} = 4 + \tfrac{1!\,0!}{2!}\cdot\big(-(-2)\big) = 4+1 = 5 \qquad \text{(regola 3; negativo nella 4)}$$

Somma $2{,}5+3{,}5+5 = 11 = v(N)$. Il calcolo esponenziale sulla funzione caratteristica della tabella dà gli stessi valori.

Approfondimento (l'aeroporto con n regole). Con le formule generali il gioco dell'aeroporto torna compatto. La dimostrazione del teorema di Littlechild e Owen è già una MC-Net: per ogni tratto j la regola

$$(x_j \vee x_{j+1} \vee \cdots \vee x_n) \to c_j - c_{j-1}$$

("almeno uno di quelli che hanno bisogno del tratto j"). Sono n regole invece di 2^n-1; verificato al calcolatore che riproducono v. La scelta del linguaggio delle formule decide quanto è compatta la rappresentazione.

Riassunto MC-Nets. Rappresentazione compatta e **universale**; il valore di una coalizione si calcola in modo efficiente; il valore di Shapley si calcola in tempo **lineare** nella dimensione dell'insieme delle regole.

6.5 Approssimazione Monte Carlo del valore di Shapley

L'idea Monte Carlo. Se si vuole conoscere una probabilità $P(x)$ ma la si sa solo campionare, la si approssima con le frequenze: si genera un campione D di dimensione M da $P(X)$ e si stima

$$P_D(X = x) = \frac{M_{X=x}}{M},$$

dove $M_{X=x}$ è il numero di volte in cui è uscito x.

Applicata a Shapley. Il valore di Shapley è una *media* del contributo marginale su tutte le $n!$ permutazioni, cioè il valore atteso del contributo marginale di i quando la permutazione è scelta uniformemente a caso. Invece di sommare su tutte, se ne campionano m:

$$\tilde{\phi}(i, v) = \frac{1}{m} \sum_{\pi \in \mathcal{P}} \Big(v\big(B(\pi, i) \cup \{i\}\big) - v\big(B(\pi, i)\big) \Big),$$

dove $\mathcal{P} \subset \Pi_N$ è un insieme di m permutazioni estratte a caso.

```
# Input: v funzione caratteristica; m numero di campioni
def MC_Shapley(v, m):
    φ̃_i = 0 for each i in N
    for k = 1 ... m:
        π_k = random permutation of N
        for i = 1 ... n:
            sv = v(B(π_k, i) ∪ {i}) - v(B(π_k, i))
            φ̃_i += sv
    for i = 1 ... n:
        φ̃_i = φ̃_i / m
    return φ̃_1, ..., φ̃_n
```

Attenzione (indice nello pseudocodice delle slide). Nell'ultimo ciclo le slide scrivono "**for** $k = 1 \dots n$" ma poi dividono $\tilde{\phi}_i$: l'indice giusto è i, come sopra.

Proprietà.

- **Calcolo efficiente**: m permutazioni, n valutazioni di v ciascuna; se v si calcola in tempo polinomiale, tutto è polinomiale.
- L'**errore** diminuisce come $1/\sqrt{m}$: per dimezzarlo servono quattro volte tanti campioni.
- Molto efficace in pratica; si applica a *qualunque* gioco e a qualunque rappresentazione.
- Esistono tecniche di campionamento che riducono la varianza più in fretta: *antithetic sampling*, estensione multilineare, metodi kernel (riferimento: Mitchell et al., 2022, *Sampling Permutations for Shapley Value Estimation*, JMLR).

Approfondimento (perché $1/\sqrt{m}$, e una garanzia esplicita). Ogni permutazione dà un contributo marginale X_k indipendente, con media esatta ϕ_i: la stima è *non distorta*. La sua varianza è σ_i^2/m, quindi l'errore tipico (deviazione standard) è $\sigma_i/\sqrt{m}$. Se i contributi marginali stanno in un intervallo di ampiezza r, la disuguaglianza di Hoeffding dà

$$P\big(|\tilde{\phi}_i - \phi_i| \geq \varepsilon\big) \leq 2\exp\left(-\frac{2m\varepsilon^2}{r^2}\right),$$

cioè con $m \geq \frac{r^2}{2\varepsilon^2} \ln \frac{2}{\delta}$ campioni l'errore è sotto ε con probabilità almeno $1-\delta$. È la "garanzia sulla soluzione" promessa all'inizio della lezione; le slide dicono invece che non c'è modo di fissare m esattamente e che in pratica $m \in [1000, 10000]$ funziona bene.

6.6 Riepilogo delle tecniche

Tecnica	Pro	Contro
Classi speciali (aeroporto)	rappresentazione compatta; Shapley veloce	espressività limitata
Monte Carlo	Shapley veloce; proprietà di convergenza; si applica a qualunque gioco e rappresentazione	non esatto; non c'è un modo esatto di scegliere m (in pratica 1000–10000)
ISG	rappresenta molti giochi; Shapley veloce; poca memoria	non completo: non rappresenta qualunque gioco
MC-Nets	completo; Shapley veloce; di solito poca memoria	memoria $O(2^n)$ nel caso peggiore; sposta il lavoro sul calcolo dei contributi marginali, che vanno precalcolati

6.7 Laboratorio: Aeroporto, Monte Carlo, ISG e MC-Nets

LAB

[CoalitionalGamesComputationalIssues_Blank.ipynb, Lab06-Coalitional_Game_Theory_Computational_Issues.pdf]

Obiettivo: implementare le tre strade della teoria e verificare ogni volta che il risultato coincide con la formula esponenziale della lezione 5. Il notebook è vuoto (non ci sono output attesi): tutte le soluzioni qui sotto sono state scritte ed eseguite, e i numeri riportati sono misurati. Le slide del lab sono un sottoinsieme di quelle della lezione.

Il notebook riparte dal codice della lezione 5, con una differenza: ora `powerset` aggiunge anche `frozenset()` con valore 0, quindi la formula di Shapley può leggere `v[C - {i}]` senza `KeyError` (il problema segnalato nella lezione 5).

6.7.1 Il gioco dell'aeroporto

Il notebook racconta l'origine del problema (Littlechild e Owen, 1973) con un costo di €1000 al km e piste da 8, 11, 13, 18 km. La funzione caratteristica è già fornita:

```python
def airport_game_characteristic_function(cost_list):
    player_list = [str(i) for i in range(len(cost_list))]
    return {subset: (max([cost_list[int(i)] for i in subset])
                     if subset != frozenset() else 0)
            for subset in powerset(player_list)}
```

Da scrivere: Shapley con la formula dedicata. La versione lineare accumula la quota del tratto corrente:

```python
def airport_game_shapley_value(cost_list):
    c = sorted(cost_list)
    n = len(c)
    phi, acc, prev = [], 0.0, 0
    for j, cj in enumerate(c):          # restano n - j giocatori
        acc += (cj - prev) / (n - j)
        phi.append(acc)
        prev = cj
    return phi
```

Sui costi $(8000, 11000, 13000, 18000)$ restituisce $(2000, 3000, 4000, 9000)$, identico alla formula esponenziale `shapley(v)`.

> **Attenzione (un "/2" di troppo nel notebook).** Nel calcolo passo per passo il notebook scrive per il giocatore 4 "(€18000 − €13000)/2 = €5000". L'ultimo tratto lo paga solo lui: si divide per 1, e infatti il risultato €5000 è giusto.

Formula dedicata contro formula generale. Giochi casuali con costi distinti fra 1 e 99:

n	Coalizioni	Costruire v	Shapley generale	Shapley aeroporto
12	4 096	0,01 s	0,03 s	10 µs
16	65 536	0,16 s	0,79 s	19 µs
20	1 048 576	3,7 s	19,3 s	16 µs

I valori coincidono sempre. Con $n = 20$ il vettore dei costi occupa 216 byte, il dizionario 42 MB secondo `sys.getsizeof`.

> **Attenzione (`sys.getsizeof` misura solo il contenitore).** `getsizeof` non conta gli oggetti contenuti: per il dizionario misura la tabella hash ma non il milione di `frozenset` che fanno da chiavi, quindi la memoria vera è molto più grande di 42 MB. Lo stesso vale più avanti per il grafo: `getsizeof(G)` restituisce 48 byte qualunque sia il grafo. Il confronto del notebook va preso come indicativo.

6.7.2 Approssimazione Monte Carlo

Prima versione, ingenua: si parte dalla funzione caratteristica completa.

```python
def permutation_sampling(player_list, m):
    return [random.sample(player_list, len(player_list)) for _ in range(m)]

def predecessors(player, player_list, permutation):
    return frozenset(permutation[:permutation.index(player)])

def shapley_montecarlo(v, m):
    players = list(max(v))
    phi = {p: 0.0 for p in players}
    for perm in permutation_sampling(players, m):
        for p in players:
            B = predecessors(p, players, perm)
            phi[p] += v[B | {p}] - v[B]
    return {p: phi[p] / m for p in players}
```

Sull'aeroporto con 20 giocatori e $m = 1000$ impiega 0,09 s (contro 19 s dell'esatto), con un errore medio percentuale (MAPE) del 9,3%.

> **Attenzione (il test del notebook ordina le stime).** La cella di test confronta le stime *ordinate per valore* con i valori esatti. Funziona solo perché nell'aeroporto, con costi distinti, ϕ_i cresce con i; ma ordinando si riassegnano le stime ai giocatori sbagliati quando due sono vicine, e l'errore risulta *più basso* del vero: sullo stesso campione 6,8% ordinando, 9,3% confrontando giocatore per giocatore. Il confronto corretto è per nome del giocatore.

Versione migliore. Il notebook chiede di non passare la funzione caratteristica completa (2^n valori: già costruirla costa più che calcolare Shapley esatto con la formula dedicata). Nell'aeroporto i contributi marginali di una permutazione si calcolano in un'unica passata: chi entra paga la differenza fra il proprio costo e il massimo raggiunto finora.

```python
def airport_mc_fast(cost_list, m):
    n = len(cost_list)
```

```
    phi = [0.0] * n
    idx = list(range(n))
    for _ in range(m):
        random.shuffle(idx)
        cur = 0                          # pista costruita finora
        for i in idx:
            if cost_list[i] > cur:
                phi[i] += cost_list[i] - cur
                cur = cost_list[i]
    return [x / m for x in phi]
```

Costo $O(m \cdot n)$, senza mai costruire v.

Esperimento facoltativo: errore contro m. MAPE medio su 20 ripetizioni, costi distinti fra 1 e 999:

m	MAPE ($n = 20$)	MAPE$\cdot\sqrt{m}$	MAPE ($n = 100$)	Tempo ($n = 100$)
10	91,1%	2,88	152,9%	0,3 ms
100	27,0%	2,70	58,9%	2,8 ms
1 000	8,0%	2,52	19,1%	33 ms
10 000	2,8%	2,84	6,2%	323 ms

La pendenza in scala log-log è $-0{,}505$ per $n = 20$ e $-0{,}467$ per $n = 100$: l'errore scende come $1/\sqrt{m}$, come dice la teoria (MAPE$\cdot\sqrt{m}$ resta circa costante). Il grafico della slide "Error reduces as sample increases" mostra la stessa retta.

> **Osservazione (l'errore relativo pesa sui giocatori piccoli).** Con $n = 20$ e $m = 1000$ il giocatore con la pista più corta ha un errore relativo del 24%, quello con la pista più lunga dell'1,1%. Il giocatore più piccolo paga poco (contribuisce solo quando arriva per primo, cioè in $1/n$ delle permutazioni), quindi il suo contributo è quasi sempre zero e raramente grande: varianza alta rispetto alla media. Per questo con $n = 100$ il MAPE è più alto a parità di m: ci sono più giocatori con quote piccole.

6.7.3 Giochi su sottografo indotto

Il notebook usa `networkx`: un grafo casuale di Erdős–Rényi con 10 nodi e 25 archi, più un cappio per nodo, con pesi interi casuali fra 1 e 100.

```
def ISG_coalition_value(graph, coalition):
    return graph.subgraph(coalition).size(weight='weight')

def ISG_shapley_value(graph):
    phi = {}
    for i in graph.nodes():
        w_ii = graph.edges[i, i]['weight'] if graph.has_edge(i, i) else 0
        phi[i] = w_ii + 0.5 * sum(graph.edges[i, j]['weight']
                                  for j in graph.neighbors(i) if j != i)
    return phi
```

Il valore del sottografo indotto è già in `networkx`: `subgraph` tiene solo gli archi interni alla coalizione (cappi compresi) e `size(weight=...)` ne somma i pesi. Il confronto con la funzione caratteristica completa (1024 coalizioni) dà valori identici; la formula ISG impiega 25 µs, quella esponenziale 9 ms. Sull'esempio della teoria si ritrovano $v(\{1,2,4\}) = 7$, $v(\{1,2,3,4\}) = 20$ e $\phi = (5,5,6{,}5,5{,}5,2)$.

> **Attenzione (con i cappi, `neighbors(i)` contiene i stesso).** In `networkx` un nodo con un cappio è vicino di se stesso. Scrivendo la formula alla lettera, $w_{ii} + \frac{1}{2}\sum_{j \in \texttt{neighbors}(i)} w_{ij}$, il cappio viene contato una volta e mezza. Sul grafo del lab la somma dei valori ingenui è 1920,5 invece di $v(N) = 1633$: l'efficienza fallisce, e la cella di confronto del notebook non stampa il messaggio "The Shapley values are the same!". Serve il filtro `if j != i`.

> **Attenzione (quanti archi può avere il grafo).** Un commento del notebook dice che un grafo non orientato con N nodi ha al più $N(N+1)/2$ archi senza cappi, e che con 10 nodi e i cappi si arriva a 65. I conti giusti sono $N(N-1)/2 = 45$ senza cappi e $N(N+1)/2 = 55$ con i cappi. Il conteggio successivo "35 archi" (25 più 10 cappi) è invece corretto.

6.7.4 MC-Nets

Il notebook implementa le MC-Nets nella forma base: una lista di coppie (pattern, valore), con il pattern come insieme di giocatori in congiunzione. L'esempio è il gioco dell'aeroporto con costi $(5, 7, 10)$, che come ISG non si può rappresentare:

```python
MC_net = [({"A"}, 5), ({"B"}, 7), ({"C"}, 10),
          ({"A", "B"}, -5), ({"A", "C"}, -5), ({"B", "C"}, -7),
          ({"A", "B", "C"}, 5)]

def MC_net_coalition_value(MC_net, coalition):
    return sum(x for P, x in MC_net if set(coalition).issuperset(P))

def MC_net_shapley_value(MC_net):
    phi = {}
    for P, x in MC_net:
        for i in P:
            phi[i] = phi.get(i, 0) + x / len(P)
    return phi
```

Risultati: $v(\{A,C\}) = 5 + 10 - 5 = 10$; la funzione caratteristica ricostruita è quella dell'aeroporto ($v = 10$ per ogni coalizione che contiene C, $v(\{A,B\}) = 7$); il valore di Shapley è (1,67; 2,67; 5,67) con tutti e tre i metodi (MC-Net, formula esponenziale, formula dell'aeroporto). L'esempio a due regole della teoria dà $\phi_a = 2{,}5$, $\phi_b = 4{,}5$.

> **Attenzione (`max(MC_net)` trova i giocatori solo per caso).** `from_MC_net_to_CFG` ricava l'insieme dei giocatori con `max(MC_net)[0]`. Le regole sono tuple (pattern, valore) e le tuple si confrontano sul primo elemento: per gli insiemi `>` vuol dire "sovrainsieme proprio", come per `max(v)` nella lezione 5. Funziona perché la MC-Net del notebook ha una regola con *tutti* i giocatori. Senza quella regola si perdono giocatori senza alcun errore: con le regole $\{A,B\} \to 5$, $\{B\} \to 2$, $\{C\} \to 1$ il risultato è $\{A,B\}$ e C sparisce. Il modo robusto è l'unione dei pattern: `set().union(*(P for P, _ in MC_net))`.

In più: MC-Nets con letterali negativi. Una regola diventa una terna (positivi, negativi, valore), e si applica se `pos <= S and not (neg & S)`. Con la formula di Shapley generale della teoria, sull'esempio $x_1 \wedge x_2 \to 5, x_2 \to 2, x_3 \to 4, x_2 \wedge \neg x_3 \to -2$ si ottiene (2,5; 3,5; 5), identico al calcolo esponenziale sulla funzione caratteristica ricostruita, che coincide con la tabella delle slide.

Riepilogo della lezione

- La funzione caratteristica ha 2^n valori e Shapley li usa tutti: servono scorciatoie.
- **Aeroporto**: $v(S) = \max_{i \in S} c_i$. Shapley: $\phi_i = \sum_{j \le i} \frac{c_j - c_{j-1}}{n-j+1}$, ogni tratto di pista diviso fra chi lo usa. Si dimostra per additività; si calcola in $O(n)$.
- **ISG**: $v(C)$ è la somma dei pesi degli archi interni a C; $\phi_i = w_{ii} + \frac{1}{2} \sum_{j \in \Gamma(i)} w_{ij}$. Non completo (solo interazioni a coppie).
- **MC-Nets**: regole pattern $\to$ valore; $v(S)$ è la somma delle regole soddisfatte. Con congiunzioni: $\phi_i = \sum x/|\varphi|$. Con letterali negativi: $\frac{(p-1)!\,n!}{(p+n)!} x$ ai positivi, $\frac{p!\,(n-1)!}{(p+n)!}(-x)$ ai negativi. Complete, ma $O(2^n)$ regole nel caso peggiore.
- **Monte Carlo**: media dei contributi marginali su m permutazioni casuali; stima non distorta, errore $\propto 1/\sqrt{m}$; vale per qualunque gioco.
- Il filo comune: *additività* del valore di Shapley (scomporre il gioco in pezzi semplici) oppure *campionamento*.
- **Lab**: aeroporto lineare contro esponenziale (16 µs contro 19 s con $n = 20$), Monte Carlo con pendenza misurata $-0{,}5$, ISG con `networkx`, MC-Nets; tutto verificato contro la formula esatta.